



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

Interrogación 2

IIC2513 Tecnologías y aplicaciones WEB

19 de noviembre de 2025 17:30 hrs

Instrucciones:

- La evaluación dura 2 horas.
- **Durante la evaluación solo se puede utilizar el material entregado en sala.**
- La prueba debe ser respondida con lápiz pasta y letra legible.
- En cada respuesta debe indicar que pregunta está respondiendo.
- **Cada hoja debe tener su nombre, apellido y número de alumno.**
- **Debe escanear su prueba y subirla a canvas antes de abandonar la sala.**
- **Se descontarán 0.5 décimas a la nota final si no se siguen las instrucciones.**

Puntos por pregunta:

- Pregunta 1 — **90 pts**
- Pregunta 2 — **90 pts**
- Pregunta 3 — **40 pts**
- Pregunta 4 — **90 pts**
- Pregunta 5 — **60 pts**
- Pregunta 6 — **60 pts**
- Pregunta 7 — **90 pts**
- Pregunta 8 — **80 pts**

Puntaje total (sin bonus): 600 puntos.



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

Pregunta 1 Cookies, Session, JWT, OAuth (90 puntos)

La universidad quiere modernizar el sistema de login de Siding. Actualmente usan sesiones de servidor con cookies y se quiere migrar a JWT y también está la idea de integrar OAuth para permitir login usando Google.

a) (30 puntos)

Explique las diferencias entre Cookies de sesión, Sesiones en servidor y JWT como mecanismo de autenticación. Incluya:

1. Dónde se guarda el estado
2. Qué viaja en cada request
3. Riesgos y mitigaciones (recuerde entre otras cosas: fechas, cabeceras, etc.)

b) (20 puntos)

Recomiende un diseño híbrido indicando cuándo usar sesiones basadas en cookies y cuándo JWT.

Justifique según seguridad y escalabilidad.

c) (40 puntos) En un request a `/api/mis-cursos`, ¿cómo autentica el backend a un usuario con JWT?

Describe paso a paso el proceso que ocurre en el middleware específicamente en la llegada y validación del JWT.



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

Pregunta 2 API REST, HTTP, CRUD, Códigos de estado **(90 puntos)**

La facultad quiere desarrollar una API REST para administrar los cursos. Se necesitan operaciones CRUD sobre **/cursos**.

a) (20 puntos)

Explique que significa que la API REST sea stateless y cómo esto influye en el diseño del backend.

b) (40 puntos)

Proponga el *diseño de rutas y verbos HTTP correctos* (o sea endpoints) para:

1. Crear un curso
2. Listar todos los cursos
3. Editar el nombre del curso
4. Eliminar un curso

Declare explícitamente los supuestos que usó para responder.

c) (30 puntos)

Dado el siguiente request de un alumno:

GET /cursos/999

El curso no existe. Como diseñador de la API, ¿qué status code (código) retornaría y qué estructura JSON mínima sugeriría? (si no sabe el código exacto, al menos indique el rango numérico del error). Justifique brevemente la elección de dicho código.



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

Pregunta 3 MVC (40 puntos)

Se necesita separar responsabilidades usando el patrón **MVC**. El equipo entiende los controladores, pero tiene dificultades para comprender el ciclo completo.

a) (20 puntos)

Explique qué rol tiene cada parte del patrón **MVC** en una aplicación cliente-servidor donde el navegador solicita HTML renderizado (no considere SPA).

b) (20 puntos)

1. ¿Qué es SPA? Explique con sus palabras
2. Entregue un ejemplo donde la recomendación es utilizar SPA en vez de páginas estáticas HTML

Pregunta 4 DOM, JS, UX/UI (90 puntos)

Los estudiantes inscriben ramos en una interfaz web con botones de nombre “Curso+” y “Curso-”. Actualmente la interfaz es poco clara y se generan errores con clicks repetitivos.

a) (40 puntos)

1. Explique el rol del **DOM**, los **event listeners** y el concepto de **render tree** en la interacción del usuario con la página.
2. Cite dos ventajas de separar HTML, CSS y JS.

b) (20 puntos)

1. Proponga dos mejoras de UX/UI basadas en principios como consistencia, retroalimentación, accesibilidad o reducción de errores.



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

2. Si quiere averiguar de accesibilidad, ¿a qué fuente oficial en la web recurriría?

c) (30 puntos)

Explique la diferencia entre DOM, CSSOM y Render Tree, y cómo se forman durante la carga de una página web.

Pregunta 5 bcrypt, Política de contraseñas (60 puntos)

El equipo quiere asegurar que los estudiantes creen contraseñas robustas en su nueva plataforma.

a) (40 puntos) (recuerden el archivo “EjemplosBcrypt”)

Explique:

1. Qué es bcrypt
2. Para qué sirve el *salt*
3. Qué significa *salt rounds* y su impacto en seguridad y desempeño
4. Por qué el hash es irreversible

b) (20 puntos)

Diseña una **política de contraseñas** adecuada para estudiantes (longitud, caracteres especiales, evitar información personal, etc.)

Pregunta 6 Evaluación de un stack tecnológico (60 puntos)

Un grupo de alumnos deben desarrollar un sistema de tutorías y deben elegir un stack tecnológico adecuado. Recomiende un stack tecnológico completo para los estudiantes, justificando decisiones.

a) (20 puntos)

Entregue los supuestos coherentes sobre:



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

Cantidad de alumnos potenciales, cantidad de usuarios concurrentes, seguridad, disponibilidad de la aplicación, criticidad de la misma, aspectos de confidencialidad de la información.

b) (20 puntos)

Basado en lo anterior, ¿qué tecnologías implementaría en el backend?

c) (20 puntos)

Su jefatura le solicita que para el frontend utilice algún framework basado en Python. ¿Qué le respondería? Justifique brevemente

Pregunta 7 React (90 puntos)

El equipo del curso debe construir un componente React de `<CursoCard>` que muestre información de un curso (nombre, profesor y cupos disponibles). Además, la página principal debe permitir marcar un curso como “favorito” haciendo clic en un botón dentro de la tarjeta. Para esto se utilizará JSX, props y el hook useState.

a) (40 puntos)

Explica brevemente:

1. ¿Qué es JSX, por qué se usa y cómo se transforma internamente antes del render?
2. ¿Qué son los props y por qué en React deben considerarse inmutables?
3. ¿Qué es un hook en React y cuál es su propósito principal en componentes funcionales?
4. ¿Qué hace useState y en qué momento React vuelve a renderizar el componente?



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

b) (50 puntos)

Imagina el componente `<CursoCard>` que recibe por props:

```
{ nombre, profesor, cupos }
```

Y un componente padre `<ListaCursos>` que muestra muchas tarjetas.

Describe (con palabras, con código, mezclando ambos, con un flujo, como mejor crea) cómo diseñaría la comunicación entre padre e hijo para que al presionar el botón “Favorito”:

1. el hijo notifique al padre,
2. el padre actualice su estado (useState),
3. y React vuelva a renderizar las tarjetas destacando cuáles son favoritas.

No es necesario que escriba código, solo explique el paso a paso. Si quiere entregar código no hay problema, no seremos estrictos con la sintaxis, sino con la lógica.

Pregunta 8 Vue.js: (Idéntico a lo visto en clases 😊) (80 puntos)

Una aplicación web con Vue.js 3 utiliza componentes SFC para mostrar información dinámica. Han aparecido dos errores en el desarrollo:

1. Un desarrollador intenta acceder a variables reactivas dentro del hook `beforeCreate()`, lo que provoca comportamientos inesperados.
2. Otro desarrollador implementó un “reloj en vivo” (que actualiza la hora cada segundo) en el hook `onUpdated()`, lo que generó un comportamiento incorrecto y un uso excesivo de recursos.



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

a) (40 puntos)

Explica qué ocurre durante el ciclo de vida del componente en las fases **beforeCreate** y **created**, y por qué no se puede acceder a las variables reactivas dentro de **beforeCreate()**.

Incluye en tu explicación:

1. ¿Qué inicializa el framework antes y después de ese hook?
2. ¿Qué elementos NO existen todavía?
3. ¿Por qué el componente no tiene acceso aún a `data()` ni a la reactividad?

b) (40 puntos)

En un componente SFC que muestra un reloj, el desarrollador usó `onUpdated()` así:

```
onUpdated(() => {  
  intervalo = setInterval(actualizarHora, 1000)})
```

El resultado fue la creación de múltiples intervalos, el navegador se volvió lento y la hora comenzó a actualizarse de manera errática.

Explica paso a paso:

1. ¿Por qué `onUpdated()` es incorrecto para instalar un temporizador.?
2. ¿Por qué el comportamiento observado ocurre (relación entre actualizaciones reactivas y múltiples renders).?
3. ¿Por qué **`onMounted()`** es el hook correcto para un reloj.?
4. ¿Cómo debería comportarse el componente correctamente usando `onMounted`.?



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

No se pide escribir código, solo explicar el flujo

PREGUNTAS BONUS (Son tres, puede elegir solo dos.)

Reglas para el Bonus:

El puntaje se puede agregar a esta interrogación o si prefiere, puede solicitar que el bonus se agregue a alguna tarea.

El puntaje NO se puede fraccionar. Es decir, si elige una tarea, todos los puntos van para la tarea o bien, todos para esta interrogación.

Cada pregunta bonus vale medio punto (0,5 décimas) por lo tanto usted puede tener hasta un punto. Por ejemplo si la calificación que usted elige (tarea o interrogación) es un 5.5, la nota puede subir hasta un máximo de 6.5

No hay límite para el bonus, es decir si tiene todo bueno en esta Interrogación, incluso los bonus, puede sacarse un 8 😊

SOLO puede seleccionar dos de las tres preguntas BONUS

PREGUNTA Bonus 1 — React (useEffect + Virtual DOM) (0.5 puntos)

Una aplicación web cliente-servidor utiliza React para renderizar el listado de cursos disponibles. Al abrir la página, la aplicación debe:

1. Hacer un fetch a /api/cursos.
2. Guardar la respuesta en el estado del componente.
3. Renderizar la lista en pantalla.

El desarrollador implementó esto usando useEffect, pero no comprende cuándo y por qué React actualiza el DOM.

a) Explique con sus palabras:



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

1. ¿Cuál es el propósito del Virtual DOM y por qué React no actualiza el DOM real inmediatamente ante cada cambio?
2. ¿Qué significa la estrategia memoization en React y para qué sirve?
3. ¿Para qué sirve un hook en React?
4. ¿Qué hace exactamente useEffect y cuándo se ejecuta?

b) Describa cómo debería estructurarse el useEffect que hace la petición al backend. Indique:

- 1) ¿Qué dependencia(s) debe tener y por qué.?
- 2) ¿Qué riesgo existe si se deja el arreglo de dependencias vacío, o si se omite deliberadamente.?
- 3) ¿Qué riesgo existe si se colocan dependencias incorrectas (posible bucle de renders).?

c) Analice por qué el siguiente pseudocódigo provoca múltiples fetches:

```
useEffect(() => {  
  fetch("/api/cursos")  
    .then(r => r.json())  
    .then(data => setCursos(data));  
}, [cursos]);
```

Explique paso a paso por qué este código provoca un ciclo infinito de renders, relacionándolo con:

- ¿cómo funciona useState?
- ¿cuándo vuelve a renderizar React?



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

- ¿cómo detecta useEffect cambios en dependencias?

PREGUNTA Bonus 2 Vue 3 (ref + fetch + reactividad) (0.5 puntos)

Un componente Vue 3 debe cargar información de asignaturas desde `/api/asignaturas`. El desarrollador usa Composition API, pero no entiende bien la diferencia entre `ref()`, acceder a `.value` y cómo la reactividad actualiza la UI tras un `fetch`.

a) Explique con sus palabras (se dijo en clases y está en el código de ejemplo)

1. ¿Qué es un `ref()` en Vue 3 y por qué su valor debe accederse usando `.value`?
2. ¿Cómo Vue convierte un ref en un objeto reactivo y qué eventos del ciclo de vida disparan la actualización del DOM.?

b) Describa cómo diseñaría un proceso de carga de datos (via fetch) dentro del hook `onMounted()` usando refs. Explique (esto está en el ejemplo Lorem):

1. ¿qué variables declararías con ref?
2. ¿cuál variable guardaría los datos?
3. ¿cuál variable guardaría el estado de “cargando”,
4. ¿cómo la reactividad actualiza automáticamente la interfaz cuando la respuesta llega. (No se pide código, solo el flujo.)



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

c) Un estudiante implementó esto:

```
const asignaturas = ref([]);  
fetch("/api/asignaturas")  
  .then(r => r.json())  
  .then(data => asignaturas = data);
```

Sin embargo, el componente no funciona como se esperaba: los datos no se muestran en pantalla y la reactividad no ocurre.

Explique:

¿Por qué esta implementación falla?

¿Cuál sería la forma correcta de implementarlo?

PREGUNTA BONUS 3 ORM (0.5 puntos)

Un backend construido con Node, Koa y Sequelize debe manejar estudiantes en una base de datos. El equipo entiende cómo usar `findAll`, pero no entiende bien qué hace el ORM, cómo funciona el mapeo, ni por qué aparecen errores de sincronización cuando modifican los modelos.

a) Explique:

1. ¿Qué es un **ORM** y cuál es su rol en una aplicación que interactúa con una base de datos?
2. ¿Qué significa que un modelo en Sequelize representa el mapeo entre una tabla y una clase/objeto en el código?



Pontificia Universidad Católica de Chile
Escuela de ingeniería
Departamento de ciencias de la computación
Profesor: Hernán Cabrera

3. La diferencia conceptual entre `define()`, `sync()`, `create()` y `findOne()` dentro del ciclo de trabajo de un ORM.

b) Describa el flujo arquitectónico adecuado para crear un nuevo estudiante. Considere:

1. El rol de la capa de servicio en la validación.
2. El uso del modelo para realizar la inserción.
3. La comunicación con la capa de controlador.
4. El manejo de errores propios del dominio o la base de datos.

(No se pide código, solo el flujo arquitectónico. Puede inspirarse en su proyecto para explicar esto)

c) El equipo modificó el modelo “Estudiante” agregando un nuevo campo (columna) *email*, pero obtienen errores al iniciar el servidor:

1. “column email does not exist”
2. “DatabaseError: relation does not exist”

Explique por qué ocurren estos errores y qué conceptos del ORM están involucrados. Considere en su respuesta:

1. La relación entre la definición del modelo y el estado real de la base de datos.
2. La necesidad de mecanismos de sincronización o migraciones.
3. Los riesgos del uso de opciones como `force: true` y cómo evitar la pérdida de datos.